Warehouse Robot Path Search

A robot moves from `S` to `G` on a grid (up, down, left, right). This program finds a path with two search algorithms and compares them.

- **DFS** (uninformed): explores as deep as it can before backtracking. It finds *a* path, not necessarily the cheapest.
- **A*** (informed): uses the Manhattan distance to the goal as a heuristic. It finds the cheapest path.

No path-finding libraries are used (only `heapq` and `time` from the standard library).

Map symbols

Symbol	Meaning	Cost to enter
`S`	Start	not counted
`G`	Goal	1
`.`	Normal floor	1
`~`	Rough floor	3
`#`	Wall	cannot enter

How to run

```
python3 robot_search.py
```

Python 3 only, nothing to install.

To use a different map for the detailed run, change this line near the top of the file:

```
GRID = MAP_A
```

The three maps (`MAP_A` , `MAP_B` , `MAP_C`) are defined at the top. Replace them with your own maps if needed.

What it prints

For each algorithm on the chosen map:

- the algorithm name
- the path as a list of `(row, column)` cells
- the total path cost
- the map with the path marked by `*`
- the number of nodes expanded
- `No path found` if the goal can't be reached

Then a table for all three maps showing path length, path cost, nodes expanded and run time.

Notes

- Moves are tried in the order Up, Down, Left, Right. DFS pops the last one pushed, so it explores Right first. A different order can change the path DFS finds.
- Path cost is the sum of the cost of every cell entered, not counting `S`.
- Run times are tiny on small maps, so they vary from run to run.